

インディーゲーム開発者が見 る世界

筑波/SFCコラボLT 2026/06/14
by みずあめ

このスライドは

<https://mizuame.works/slides/>

で見れます

About Me

名前: 佐藤 良

年齢: 20歳

所属: 筑波大学情報学群情報科学類3年

- 高校生から委託でゲームエンジニアなど
- ネットワークもやります
- 最近IPAに客員研究員で入った
- 日本ゲームシナリオライター協会

X: @mizuameisgod

GitHub: @mizuamedesu

Web: <https://mizuame.works>



俺の人生対戦ゲームに狂わされすぎ...?



小/中で合計1500時間



中/高で合計3000時間



中/高で合計2000時間

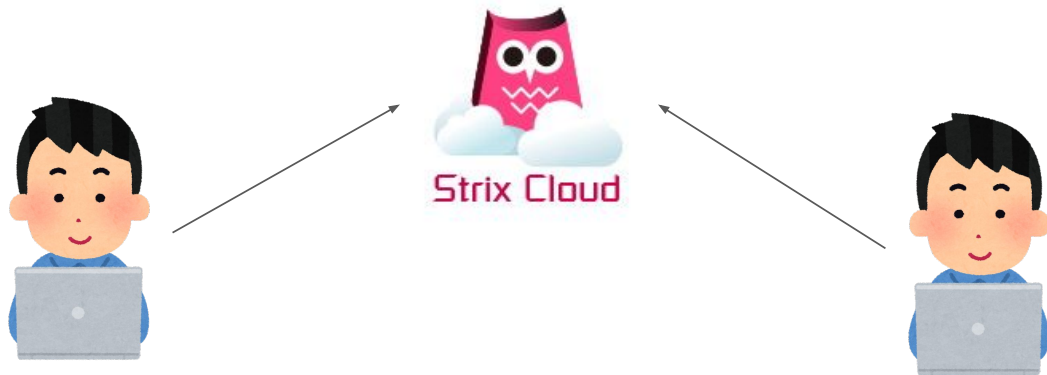
その他: Overwatch2 500時間 VALORANT 500時間 レインボーシックス シージ200時間

オンラインゲーム
を作ろう！

高校生の時にAzurePlayFab+StrixCloudでオンラインゲームを作った



リレーサーバーの限界



- リレーサーバーは権威がなく、クライアントの自己申告になる
- ワールドのステートを持つ存在がいいため、グローバルロジックが厳しい
(マイクラのようにどこかのホストを一時的にやることは可能だが、マイグレーションの問題やホスト切断時の問題が残る)
- チート耐性が皆無！(自己申告なので、はい無敵一が可能になる)

しかし！
専用サーバーは
難しすぎる

UnrealEngineの概要

- UnrealEngineはオンラインネットワーク対戦を前提とした作りのため、エンジン単体でクライアント実行バイナリと、サーバー用実行バイナリどちらも並行して作成が可能
- ある程度書き出される形式は決まっている



問題点1: そもそもIaaS/SaaSはない

- StrixCloudやPhoton Fusionなどのリレーをしてくれるもの、SteamP2P、Epic Online Services P2PなどのP2Pリレーをしてくれるものはある
- しかし、権威サーバーのバイナリをネットワーク/インフラに詳しくないゲームエンジニアが数クリックで展開できる程度の抽象化されたものがない

問題点2: クラウドVMって高すぎる

- 例えば2vCPU4RAMのEC2で1マッチ10人程度を捌けます(ゲームによるけど)
- この場合月8000円くらい
- 例えば、ユーザーが1000人なら80万、1万人なら800万とかかかる

→こんなことをやっていたらそりゃ金がなくなる

オンプレが割と現実的である

- オンラインゲームは基本的に一番計算資源が必要なゲーム本体のサーバーと、ユーザーのデータ(コインとか累積マッチ戦績とか)は分離されている
- ゲーム本体のサーバーは割と落ちることが許される、マッチ中(数分~数十分)の間のみステートが保てていれば良い

→ 可用性に問題があるオンプレと都合がいい

CNDW2025で登壇



CLOUDNATIVE DAYS WINTER 2025

オンプレとGCPのハイブリットでK3s/K8s で ゲームサーバーをオーケストレーション

@CNDW2025 学生スカラシップLT
by みずあめ

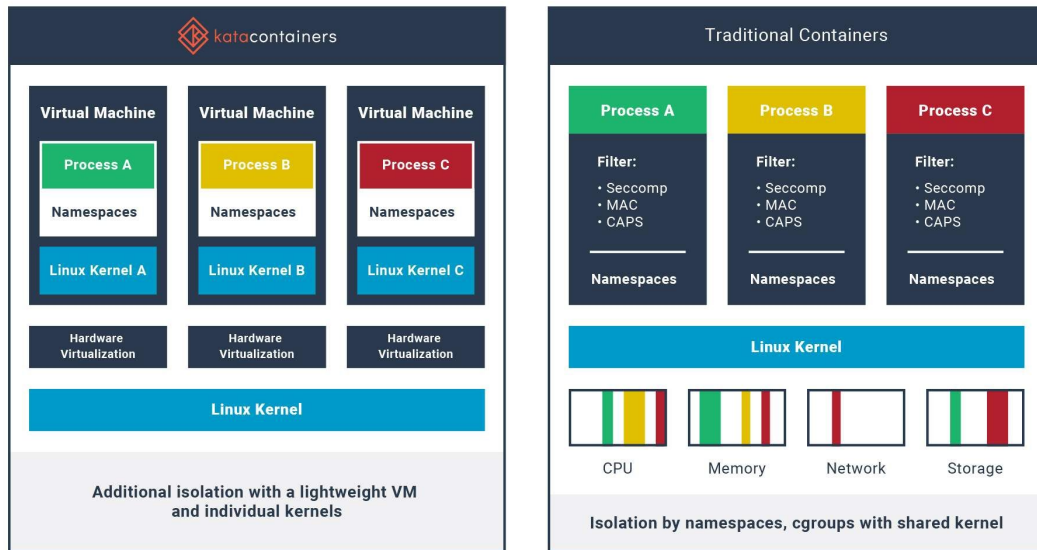


#CNDW2025

最終目標

- リレーサーバーを提供するSaaSはあるが、Dedicated serverを提供するSaaSはあまりない
- インディーゲーム界隈に、安価で提供できるDedicated server as a Serviceを作りたい
- 任意コード、バイナリの実行を許すため、セキュアな環境を作る必要がある
- 第一段階として、Kata Containersランタイムを使用したゲームランナーを構築する

Kata Containersとは



OCI コンテナ形式のランタイム。
内部では基本的にQEMUを立ち上げている。
VM並みの隔離をコンテナ環境で実行できる。

https://katacontainers.io/static/6e497f9d3752ca1e354d0d2949abc020/8fef6/katacontainers_traditionalvskata_diagram.jpg

Kata Containersとは

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: ue5-gameserver
5    namespace: game
6  spec:
7    replicas: 2
8    selector:
9      matchLabels:
10       app: ue5-server
11  template:
12    metadata:
13      labels:
14       app: ue5-server
15    spec:
16      runtimeClassName: kata
17    nodeSelector:
18      hardware: game-runner
19
20    affinity:
21      podAntiAffinity:
22        preferredDuringSchedulingIgnoredDuringExecution:
23          - weight: 100
24            podAffinityTerm:
25              labelSelector:
26                matchExpressions:
27                  - key: app
28                    operator: In
```

```
kata > ! runtimeclass.yaml
1  apiVersion: node.k8s.io/v1
2  kind: RuntimeClass
3  metadata:
4    name: kata
5  handler: kata
```

Alt runc
ランタイムクラスでkataを指定
するだけでok

Ping : 20 FPS : 183

matched

status : Debug

1

1.0/100.0

10
10



1



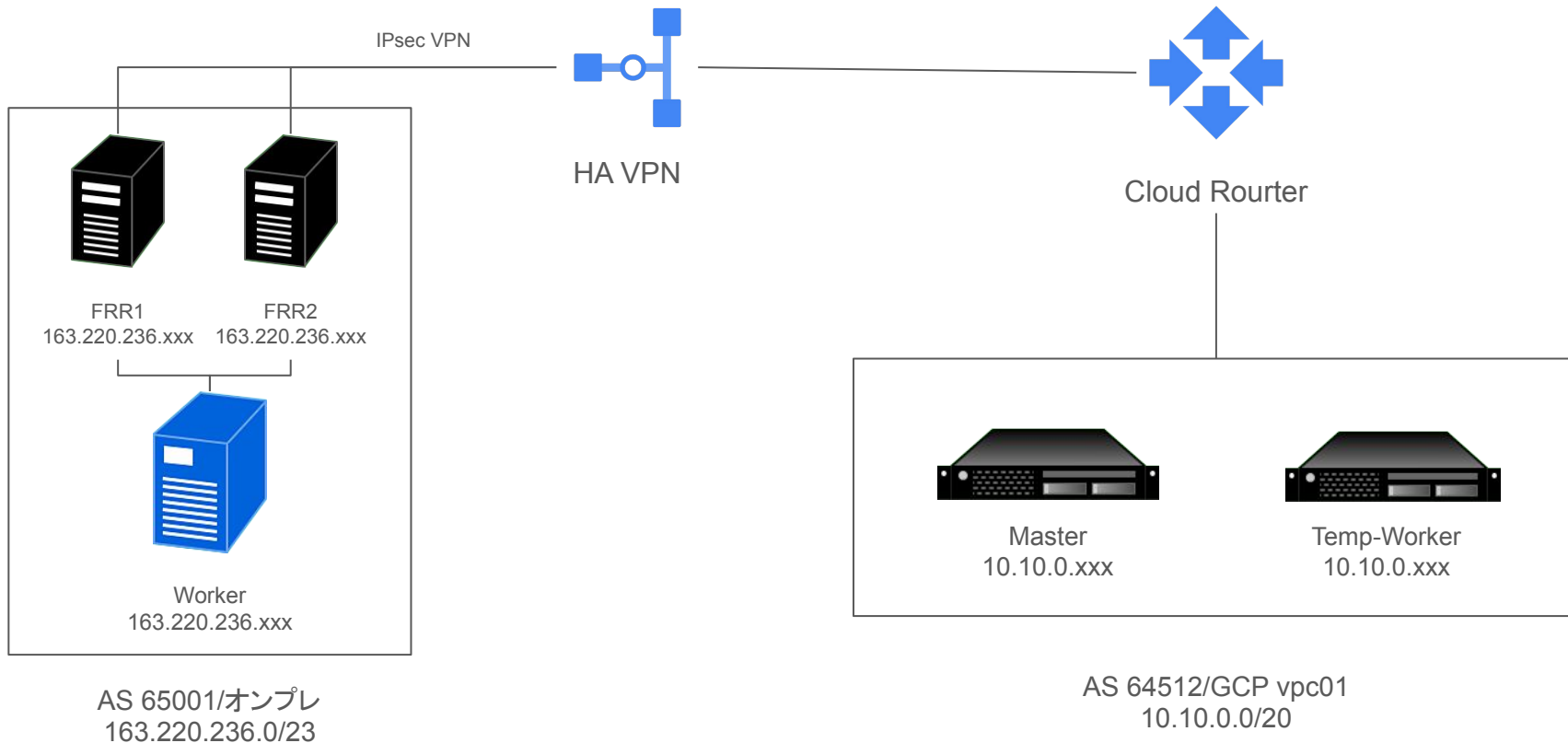
R



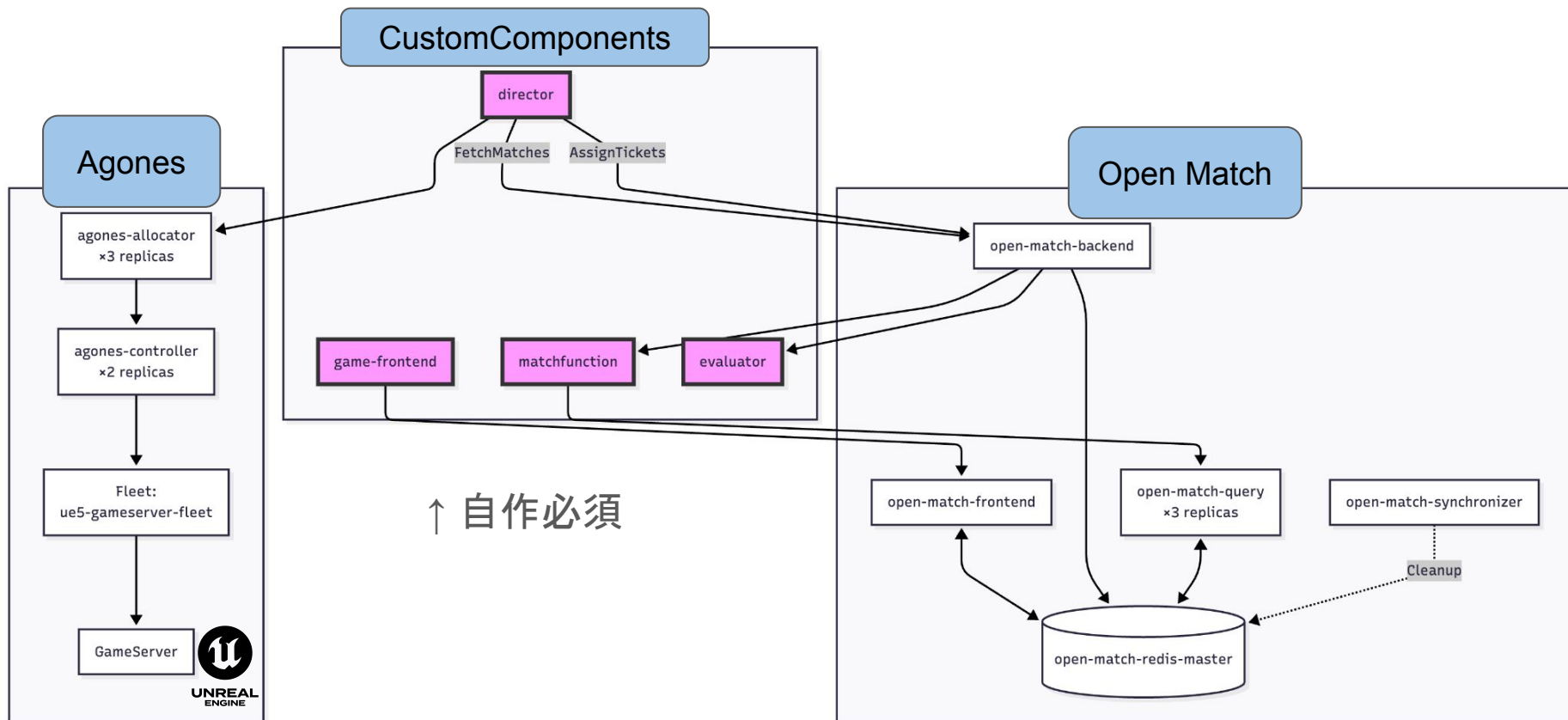
Q

X

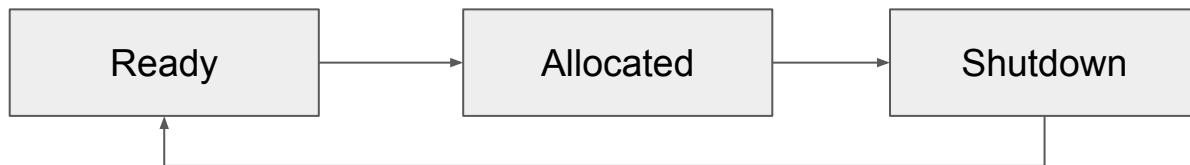
オンプレ/クラウドハイブリッドK8s



AgonesとOpen Match



Agonesの状態遷移



正確にはScheduled,Requested,Startingもある

- できる限り既存のセッションは落としたくないので、すでにAllocatedされたpodが一時VMに立っている間は、VMの削除を実施しない。
- ReadyのPodは全削除
- 全てのVMのPodが消えるか、最大許容時間を超えるとVM自体の削除を実施

闇のノードオペレーター

- onpremiseラベルを監視対象のノードに付与: 今回はuc-k8s4p
- gcp-permanentを永続ノードに付与: gcp-master

→ここに監視用オペレーターが配置される

```
mizuame@k8s-master-1:~$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
k8s-master-1.asia-northeast1-a.c.k8s-test-474912.internal	Ready	control-plane	14d	v1.31.13	10.10.0.23	<none>	Ubuntu 24.04.3 LTS	6.14.0-1018-gcp	containerd://1.7.28
uc-k8s4p	Ready	<none>	14d	v1.31.13	163.220.236.54	<none>	Ubuntu 24.04.3 LTS	6.8.0-86-generic	containerd://1.7.28


```
mizuame@k8s-master-1:~$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP	OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
gcp-temp-uc-k8s4p-1763028338	Ready	<none>	38s	v1.31.13	10.10.0.42	35.243.101.18	Ubuntu 22.04.5 LTS	6.8.0-1042-gcp	containerd://1.7.28
k8s-master-1.asia-northeast1-a.c.k8s-test-474912.internal	Ready	control-plane	14d	v1.31.13	10.10.0.23	34.146.14.104	Ubuntu 24.04.3 LTS	6.14.0-1018-gcp	containerd://1.7.28
uc-k8s4p	NotReady	<none>	14d	v1.31.13	163.220.236.54	<none>	Ubuntu 24.04.3 LTS	6.8.0-86-generic	containerd://1.7.28

```
mizuame@k8s-master-1:~$ █
```

闇のノードオペレーター

[2025-11-13 10:05:06] Node event detected: node=uc-k8s4p,
ready=False

- オンプレノードが NotReady と判断され、フェイルオーバー処理が開始される

[2025-11-13 10:05:07] Deleting completed NodeFailover resource

[2025-11-13 10:05:07] Creating NodeFailover resource for NotReady
node u

c-k8s4p

[2025-11-13 10:05:07] NodeFailover resource created

- 新しいフェイルオーバー処理がセットアップされる

闇のノードオペレーター

[2025-11-13 10:05:38] Creating GCP VM for failed node uc-k8s4p
[2025-11-13 10:05:38] Created bootstrap token
[2025-11-13 10:05:38] Successfully calculated CA certificate hash
[2025-11-13 10:05:59] Creating GCP instance gcp-temp-uc-k8s4p-1763828338
[2025-11-13 10:05:59] Successfully created instance
[2025-11-13 10:06:29] Node gcp-temp-uc-k8s4p-1763828338 has joined the cluster
[2025-11-13 10:06:29] Labeled node: node-type=gcp-temporary, node-location=gcp, hardware=game-runner

- オンプレノードが NotReady と判断され、フェイルオーバー処理が開始される。
- VMの作成が完了するとKubernetes クラスタへjoin、ラベル付与

闇のノードオペレーター

- 事前に全て(K8s、Kata Containersなど)がセットアップされたカスタムコンピュータイメージを持っている

```
gcloud compute images list --no-standard-images
```

NAME	PROJECT	FAMILY	DEPRECATED	STATUS
k8s-kata-node-v1		k8s-kata-node		READY

- このイメージを元に、VMを作成し、Joinさせる
 - kubeletに provider-id を設定することでcloud-controller-manager経由で ExternalIPを取得できる
- ```
--cloud-provider=external
--provider-id=gce://testserver-4343/asia-northeast1-b/gcp-temp-uc-k8s4p-x
xxxx
```

## 閤のノードオペレーター

- `create_bootstrap_token()`  
Kubernetes一時トークン作成
- `get_ca_cert_hash()`  
クラスタCA証明書のSHA256ハッシュ取得
- Startup Script生成  
`generate_startup_script(token, ca_hash)`  
メタデータ取得 (PROJECT, ZONE, INSTANCE\_NAME)  
kubelet設定 (cloud-provider=external, provider-id)  
kubeadm join

# 闇のノードオペレーター

```
mizuame@k8s-master-1:~$ kubectl get nodes -o wide
```

| NAME                                                      | STATUS | ROLES         | AGE | VERSION  | INTERNAL-IP    | EXTERNAL-IP | OS-IMAGE           | KERNEL-VERSION   | CONTAINER-RUNTIME   |
|-----------------------------------------------------------|--------|---------------|-----|----------|----------------|-------------|--------------------|------------------|---------------------|
| k8s-master-1.asia-northeast1-a.c.k8s-test-474912.internal | Ready  | control-plane | 14d | v1.31.13 | 10.10.0.23     | <none>      | Ubuntu 24.04.3 LTS | 6.14.0-1018-gcp  | containerd://1.7.28 |
| uc-k8s4p                                                  | Ready  | <none>        | 14d | v1.31.13 | 163.220.236.54 | <none>      | Ubuntu 24.04.3 LTS | 6.8.0-86-generic | containerd://1.7.28 |

```
mizuame@k8s-master-1:~$ kubectl get nodes -o wide
```

| NAME                                                      | STATUS   | ROLES         | AGE | VERSION  | INTERNAL-IP    | EXTERNAL-IP   | OS-IMAGE           | KERNEL-VERSION   | CONTAINER-RUNTIME   |
|-----------------------------------------------------------|----------|---------------|-----|----------|----------------|---------------|--------------------|------------------|---------------------|
| gcp-temp-uc-k8s4p-1763028338                              | Ready    | <none>        | 38s | v1.31.13 | 10.10.0.42     | 35.243.101.18 | Ubuntu 22.04.5 LTS | 6.8.0-1042-gcp   | containerd://1.7.28 |
| k8s-master-1.asia-northeast1-a.c.k8s-test-474912.internal | Ready    | control-plane | 14d | v1.31.13 | 10.10.0.23     | 34.146.14.104 | Ubuntu 24.04.3 LTS | 6.14.0-1018-gcp  | containerd://1.7.28 |
| uc-k8s4p                                                  | NotReady | <none>        | 14d | v1.31.13 | 163.220.236.54 | <none>        | Ubuntu 24.04.3 LTS | 6.8.0-86-generic | containerd://1.7.28 |

- create\_instance(vm\_name, startup\_script)  
カスタムイメージ使用: k8s-kata-node-v1  
metadata.startup-script に設定  
VM起動 → スクリプト自動実行
- Ready状態まで自動でなる
- EXTERNAL-IPが振られている  
→ Agonesが自動でグローバルIP:Portを振ってくれる

## 闇のノードオペレーター

```
mizuame@k8s-master-1:~$ kubectl get gameservers -n game -o wide
```

| NAME                             | STATE     | ADDRESS       | PORT | NODE                         | AGE |
|----------------------------------|-----------|---------------|------|------------------------------|-----|
| ue5-gameserver-fleet-z2jhh-5prsk | Scheduled | 34.146.94.130 | 7254 | gcp-temp-uc-k8s4p-1763029184 | 43s |
| ue5-gameserver-fleet-z2jhh-hdjcm | Scheduled | 34.146.94.130 | 7995 | gcp-temp-uc-k8s4p-1763029184 | 43s |

```
mizuame@k8s-master-1:~$
```

Onprem node uc-k8s4p still NotReady, applying out-of-service taint

Added taint to node uc-k8s4p

Set condition TaintApplied=True for uc-k8s4p

Applied out-of-service taint to uc-k8s4p

- VMがJoinした後、オンプレに  
node.kubernetes.io/out-of-service:NoExecute(taint付与)
- GCPの一時VMに各種サーバーが再配置される(画像参照)

## 闇のノードオペレーター

[2025-11-13 10:13:56] Node uc-k8s4p has re-joined the cluster  
[2025-11-13 10:13:56] Labeled node: node-type=onprem, node-location=onprem,  
hardware=game-runner  
2025-11-13 10:13:56] Node event detected: uc-k8s4p ready=True  
[2025-11-13 10:13:56] Onprem node recovery detected: uc-k8s4p  
[2025-11-13 10:13:56] NodeFailover phase changed: Active → Recovering

- `remove_node_taint()`

復旧を検知次第、オンプレノードの out-of-service taint削除  
GameServerがオンプレに戻れるようになる

- GCP一時ノードをcordon(新規Pod配置停止)

# 闇のノードオペレーター

```
[mizuame@k8s-master-1:~$ kubectl get gameserver -n game
```

| NAME                             | STATE     | ADDRESS        | PORT | NODE     | AGE |
|----------------------------------|-----------|----------------|------|----------|-----|
| ue5-gameserver-fleet-z2jhh-5vbqv | Scheduled | 163.220.236.54 | 7398 | uc-k8s4p | 24s |
| ue5-gameserver-fleet-z2jhh-hksj7 | Ready     | 163.220.236.54 | 7555 | uc-k8s4p | 15m |

```
[mizuame@k8s-master-1:~$ kubectl get gameserver -n game
```

| NAME                             | STATE | ADDRESS        | PORT | NODE     | AGE |
|----------------------------------|-------|----------------|------|----------|-----|
| ue5-gameserver-fleet-z2jhh-5vbqv | Ready | 163.220.236.54 | 7398 | uc-k8s4p | 64s |
| ue5-gameserver-fleet-z2jhh-hksj7 | Ready | 163.220.236.54 | 7555 | uc-k8s4p | 16m |

```
mizuame@k8s-master-1:~$ █
```

[2025-11-13 10:14:01] NodeFailover phase changed: Recovering → Draining

[2025-11-13 10:14:01] Condition set: GameServersDrained=True

[2025-11-13 10:14:01] Started deletion of temporary node

gcp-temp-uc-k8s4p-1763028338

[2025-11-13 10:14:36] Deleted node gcp-temp-uc-k8s4p-1763028338

[2025-11-13 10:14:36] Deleting GCP instance gcp-temp-uc-k8s4p-1763028338

## Agonesへのシャットダウン通知実装

- AgonesSDKを使用すると、WatchGameServer()でデリゲートを取れる
- デリゲートは状態遷移が発生すると発火する。
- Allocatedが帰ってきた場合、ゲームサーバー側はクライアントが割り当てられたことを知ることができる。
- UEの場合、ゲームモードのログアウト関数の後などに、プレイヤーコントローラーの管理処理を書き、この人数が0以下になったらSDKのShutdown関数を呼ぶ
- Shutdown関数はAgonesに対し回収依頼を発行し、自動でpodがクリーンアップされる(UE自身が直接シャットダウンするわけではない)

終わりに

世の中にもっと対戦ゲームが増えると嬉しいなと思います

(TGSとかBitSummitとか東京ゲームダンジョンとか言ってもマジでインディーオンラインゲーを作ってる人がいない)